

NANYANG TECHNOLOGICAL UNIVERSITY
SCHOOL OF COMPUTING AND DATA SCIENCE



SC4003
Intelligent Agents

Assignment 2 Report

By: NICHOLAS YEO (U2323221 C)
22 April 2026

Table Of Contents

1. Introduction.....	1
2. Agent Strategy Development.....	4
2.1 Agent Core Values.....	
2.1.1 “Always Cooperate first”.....	5
2.1.2 “If everyone attacks me, I’ll retaliate”.....	6
2.1.3 “If anyone seemed highly suspicious, I’ll protect myself”.....	
2.1.4 “I’ll trust you if you show me I can”.....	
2.1.5 “Be a nice person”.....	
3. Code Implementation	4
4. Evaluation	4
6. Conclusion	4

1. Introduction

The Prisoner's Dilemma is a fundamental problem in game theory that explores the tension between cooperation and self-interest. In this assignment, we will be looking at a **three-player Prisoner's Dilemma problem** and we will be developing an agent to play against a set of different types of agents in a tournament. The objective of the agent is to maximise its average payoff across approximately **100 match rounds** and against varying opponent behaviors.

*C = cooperate, **D** = defect*

(Player A, Player B, Player C)	Payoff
(C, D , D)	0
(D , D , D)	2
(C, C, D) (C, D , C)	3
(D , C, D) (D , D , C)	5
(C, C, C)	6
(D , C, C)	8

Figure 1: Agent Payoff Chart

<i>Player A: COOPERATES (C)</i>	<i>Player C: Cooperate (C)</i>	<i>Player C: Defect (D)</i>
<i>Player B: Cooperate (C)</i>	(6, 6, 6)	(3, 8, 3)
<i>Player B: Defect (D)</i>	(8, 3, 3)	(5, 5, 0)

<i>Player A: DEFECTS (D)</i>	<i>Player C: Cooperate (C)</i>	<i>Player C: Defect (D)</i>
<i>Player B: Cooperate (C)</i>	(8, 3, 3)	(5, 0, 5)
<i>Player B: Defect (D)</i>	(5, 5, 0)	(2, 2, 2)

Figure 2: Three-Player Prisoner's Dilemma Matrix

Based on the payoff chart and the Prisoner's Dilemma Matrix given, the highest average scores for all agents is (6, 6, 6) where all agents cooperates. We also conclude that the *dominant strategy* here would be *defect* here:

$$U(D, C, D) = 5 > U(C, C, D) = 3$$

$$U(D, D, D) = 2 > U(C, D, D) = 0$$

Defect (D) takes a better payoff compared to **Cooperate (C)** regardless of the opponents actions.

2. Agent Strategy Development

The strategy adopted for this agent is a *Conditional Cooperation Strategy*, a behavioural strategy where individuals are willing to cooperate, provided that others in their group do the same. It first cooperates to encourage mutual benefits and monitors the actions of other agents progressively. By analysing short-term defection patterns, the agent is able to detect hostile environments and respond defensively when appropriate. It also incorporate a mechanism to restore cooperation if opponents show a good cooperative behaviour rate.

2.1 Agent Core Values

This design aim to balance between trust, adaptibility, and self-protection by adhering to it's core values:

1. *“Always cooperate first.”*
2. *“If everyone attacks me, I’ll retaliate.”*
3. *“If anyone seemed highly suspicious, I’ll protect myself.”*
4. *“I’ll trust you if you show me I can.”*
5. *“Be a nice person.”*

2.1.1 “Always Cooperate First.”

```
// 1. Always be kind, cooperate for the first round
if (n == 0) {
    return 0;
}
```

“Do onto others what you want others to do onto you”

In order to harness good faith from others, kindness begins with us first. On our very first round, we will start by cooperating first, and we will watch how the other agents react towards our kind nature.

2.1.2 “If everyone attacks me, I’ll retaliate.”

```
// 2. If one of the opponents defected in the last round, enter distrust mode and defect to punish
if (oppHistory1[n-1] == 1 && oppHistory2[n-1] == 1) {
    distrustMode = true;
    return 1; // Defect to punish
}
```

“If your opponent is of choleric temper, seek to irritate him.”

This is an immediate retaliation. If both opponents decide to defect against our agent, our agent will turn against them immediately and enter a state of distrust for the following rounds.

2.1.3 “If anyone seems highly suspicious, I’ll protect myself.”

```
// 3. If one opponent has been defecting frequently recently, enter distrust mode and defect to protect yourself
if( (oppHistory1[n-1] == 1 && opp1DefRate >= 0.40) || (oppHistory2[n-1] == 1 && opp2DefRate >= 0.40) ) {
    distrustMode = true;
    return 1; // Defect to punish
}
```

“All warfare is based on deception”

If one of the opponents defects, we’ll look at their recent history and evaluate their rate of deception in the past few rounds. If they have been defying against everyone a little too frequently, we’ll go with the **dominant strategy** of **defect** to protect ourselves and enter a state of distrust while we continue to watch their movements.

2.1.4 “I’ll trust you if you show me I can.”

```
// 4. If in distrust mode, check if opponents have been mostly cooperative recently to regain trust
if(distrustMode) {

    // Exit distrust mode if both opponents have been mostly cooperative recently
    if(opp1DefRate <= 0.15 && opp2DefRate <= 0.15) {
        distrustMode = false;
        return 0;
    }
    return 1; // Continue to defect in distrust mode
}
```

“Trust is earned when actions meet words.”

While in a state of **distrust**, we’ll have to monitor both opponents and if their defection rate falls below a certain rate, we’ll reward by opening up and start cooperating again. Since trust was once given and betrayed, it would take a longer time to build trust within one another. Hence, **BOTH** opponents **defection rate** must fall to or below **0.15** for our agent to indulge in cooperation again.

2.1.5 “Be a nice person”

```
// 5. Cooperate by default to preserve long-run gains
return 0;
```

“Be kind whenever possible. It is always possible.”

By default, our agent tries to cooperate with everyone as much as possible since it’s aware that “Social Welfare” requires cooperation and it’s total payoff is far greater than constant retaliation and defection.

3. Code Implementation

```
class Yeo_Nicholas_Player extends Player {  
  
    boolean distrustMode = false;  
  
    int selectAction(int n, int[] myHistory, int[] oppHistory1, int[] oppHistory2) {  
  
        // 1. Always be kind, cooperate for the first round  
        if (n == 0) {  
            return 0;  
        }  
  
        // Check recent defections in the last 5 rounds to detect strategy switches or retaliation patterns  
        int window = Math.min(a: 5, n);  
        int opp1RecentDef = 0;  
        int opp2RecentDef = 0;  
  
        for (int i = n-window; i < n; i++) {  
            if (oppHistory1[i] == 1) {  
                opp1RecentDef++;  
            }  
  
            if (oppHistory2[i] == 1) {  
                opp2RecentDef++;  
            }  
        }  
  
        double opp1DefRate = (double) opp1RecentDef / window;  
        double opp2DefRate = (double) opp2RecentDef / window;  
  
        // 2. If one of the opponents defected in the last round, enter distrust mode and defect to punish  
        if (oppHistory1[n-1] == 1 && oppHistory2[n-1] == 1) {  
            distrustMode = true;  
            return 1; // Defect to punish  
        }  
  
        // 3. If one opponent has been defecting frequently recently, enter distrust mode and defect to protect yourself  
        if( (oppHistory1[n-1] == 1 && opp1DefRate >= 0.40) || (oppHistory2[n-1] == 1 && opp2DefRate >= 0.40) ) {  
            distrustMode = true;  
            return 1; // Defect to punish  
        }  
  
        // 4. If in distrust mode, check if opponents have been mostly cooperative recently to regain trust  
        if(distrustMode) {  
  
            // Exit distrust mode if both opponents have been mostly cooperative recently  
            if(opp1DefRate <= 0.15 && opp2DefRate <= 0.15) {  
                distrustMode = false;  
                return 0;  
            }  
  
            return 1; // Continue to defect in distrust mode  
        }  
  
        // 5. Cooperate by default to preserve long-run gains  
        return 0;  
    }  
}
```

Figure 3: Code snippet of the developed agent

4. Evaluation

The agent, **Yeo_Nicholas_Player** have went up against 6 players with different strategies:

1. NicePlayer
2. NastyPlayer
3. FreakyPlayer
4. TolerantPlayer
5. RandomPlayer
6. T4TPlayer

The source code was modified to compute key performance indicators, including average rank, cumulative score, number of wins and overall rankings. These metrics enable a systematic evaluation of player performance and facilitate the identification of the top-performing player.

The number of tournaments consisting of approximately **100 match rounds** (90 to 110) each is displayed for clarity. The overall rankings are shown below:

```
=== Overall Ranking After 100 Tournaments ===
1. Yeo_Nicholas_Player | Avg Rank: 1.52 | Avg Score: 169.69 | Wins: 62
2. TolerantPlayer | Avg Rank: 2.20 | Avg Score: 164.37 | Wins: 20
3. T4TPlayer | Avg Rank: 2.40 | Avg Score: 163.61 | Wins: 18
4. NicePlayer | Avg Rank: 4.25 | Avg Score: 152.36 | Wins: 0
5. FreakyPlayer | Avg Rank: 5.20 | Avg Score: 145.37 | Wins: 0
6. NastyPlayer | Avg Rank: 5.82 | Avg Score: 140.44 | Wins: 0
7. RandomPlayer | Avg Rank: 6.61 | Avg Score: 134.51 | Wins: 0

=== Overall Ranking After 200 Tournaments ===
1. Yeo_Nicholas_Player | Avg Rank: 1.46 | Avg Score: 169.69 | Wins: 132
2. TolerantPlayer | Avg Rank: 2.23 | Avg Score: 164.56 | Wins: 40
3. T4TPlayer | Avg Rank: 2.46 | Avg Score: 163.14 | Wins: 27
4. NicePlayer | Avg Rank: 4.30 | Avg Score: 151.73 | Wins: 0
5. FreakyPlayer | Avg Rank: 5.07 | Avg Score: 145.92 | Wins: 1
6. NastyPlayer | Avg Rank: 5.89 | Avg Score: 139.96 | Wins: 0
7. RandomPlayer | Avg Rank: 6.60 | Avg Score: 133.85 | Wins: 0

=== Overall Ranking After 500 Tournaments ===
1. Yeo_Nicholas_Player | Avg Rank: 1.50 | Avg Score: 169.30 | Wins: 314
2. TolerantPlayer | Avg Rank: 2.28 | Avg Score: 164.36 | Wins: 92
3. T4TPlayer | Avg Rank: 2.33 | Avg Score: 163.93 | Wins: 93
4. NicePlayer | Avg Rank: 4.34 | Avg Score: 151.50 | Wins: 1
5. FreakyPlayer | Avg Rank: 5.12 | Avg Score: 146.19 | Wins: 0
6. NastyPlayer | Avg Rank: 5.83 | Avg Score: 140.65 | Wins: 0
7. RandomPlayer | Avg Rank: 6.59 | Avg Score: 134.42 | Wins: 0
```

Figure 4: Tournament Results

Based on **Figure 4**, our agent, **Yeo_Nicholas_Player** triumph over all players with it's Conditional Cooperation Strategy.

After 100 Tournaments				
RANK	PLAYER	AVG RANK	AVG SCORE	WINS
1	Yeo_Nicholas_Player	1.52	169.69	62
2	TolerantPlayer	2.20	164.37	20
3	T4TPlayer	2.40	163.61	18
4	NicePlayer	4.25	152.36	0
5	FreakyPlayer	5.20	145.37	0
6	NastyPlayer	5.82	140.44	0
7	RandomPlayer	6.61	134.51	0

After 200 Tournaments				
RANK	PLAYER	AVG RANK	AVG SCORE	WINS
1	Yeo_Nicholas_Player	1.46	169.69	132
2	TolerantPlayer	2.23	164.56	40
3	T4TPlayer	2.46	163.14	27
4	NicePlayer	4.30	151.73	0
5	FreakyPlayer	5.07	145.92	1
6	NastyPlayer	5.89	139.96	0
7	RandomPlayer	6.60	133.85	0

After 500 Tournaments				
RANK	PLAYER	AVG RANK	AVG SCORE	WINS
1	Yeo_Nicholas_Player	1.50	169.30	314
2	TolerantPlayer	2.28	164.36	92
3	T4TPlayer	2.33	163.93	93
4	NicePlayer	4.34	151.50	0
5	FreakyPlayer	5.12	146.19	1
6	NastyPlayer	5.83	140.65	0
7	RandomPlayer	6.59	134.42	0

The result across **100, 200 and 500 tournaments** showed a high level of consistency and convergence in player performance rankings.

Yeo_Nicholas_Player have consistently triumph over all players achieving the highest ranking across all tournament sizes, with an average rank close to **1.5** and the highest average score of **169** approximately. The number of wins have also increased proportionally with the number of tournaments indicating a strong dominance and robustness against randomness. This suggests that the strategy that was adopted was deemed effective as it balances cooperation and retaliation, allowing it to exploit cooperative environments while protecting itself from defectors.

TolerantPlayer and **T4TPlayer** is a strong competitive here. Both strategies average ranks between 2 and 3 with a high and stable scores. **TolerantPlayer** shows slightly better consistency in rank, while **T4TPlayer** achieves comparable performance with slightly more variability. Their win counts increase steadily, indicating they remain competitive but unable to consistently outperform **Yeo_Nicholas_Player** strategy.

5. Conclusion

Even though the dominant strategy is *defect* and it incurs the highest immediate payoff in a single-round setting, it's effectiveness is bound to reach to a stationary progression in the long run.

Based on our findings, it strongly suggests that adaptive strategies are necessary in a long-term optimisation in competitive environments compared to a more defensive strategy or a purely cooperative strategy like **NastyPlayer** or **NicePlayer** respectively. The inability to be adaptive to retaliate or exploit and cooperate at appropriate times incurs a lower overall payoff over time.

As the number of tournaments increases, the results exhibits the importance of a balance in cooperation and defection correlated with a strategy in mind to execute certain actions whenever appropriate.